4.0 Software Development Plan

4.1 Plan Introduction

The Software Development Plan provides the details of the planned development for the Gather CSCI which provides a web application that organizes academic information and builds a to-do list with assignments and their due dates for users.
The Gather web application will include features that collect information across various websites and pdf documents in order to put together a task list with all the assignments and their due dates. Gather will use LLM in order to scrape all the important information across various websites such as Brightspace, professor's webpages, and pdf documents - syllabuses. Once LLM collects and organizes information, users will be prompted with a Review page where they will be able to see all the collected assignments and check if the information is correct. Then they will proceed to either accept or decline certain tasks. The core development process will include detailed design and analysis of requirements, as well as frontend development using React.js, backend development using Node.js and Express.js, database establishment using Firestore, information processing by AI using Llama, documentation and final demonstration/delivery.

Project milestones:

- Project's proposal/definition - week 2-3
- Project's requirements and planning - week 5
- Development planning and environment establishment - week 7
- Core development and design - week 10
- Detailed design - week 12
- Prototype done and demonstration - week 13
- Finalizing details and user documentation - week 14
- Final Presentation and delivery - week 15-16

4.1.1 Project Deliverables

Deliverables	Description	Delivery dates
Project proposal	Getting together with a group and defining what the project will be, as well as key features.	Week 2-3
Project's requirements document and planning	The team will deliver the Requirements specification document. It will define all the requirements needed for proper function of the web application.	Week 5
Software Development Plan	SDP document will describe the planning as well as approach used to develop and manage Gather. This document will include group members' responsibilities,	Week 7

	resources, deliverables, schedule of development and it will also describe how the progress will be tracked.	
Environment Establishment	The team will establish different environments depending on each member's role.	Week 7
Software Design Description [Architecture Section]	This document will explain different components and features of Gather interact, such as frontend, backend, AI-assisted information collection, and database.	Week 10
Core development and design	Each team member will have core development of their part done. The backend core details will be established, as well as frontend's. Database will also be established by this point.	Week 10
Software Design Description Document [Detailed Section]	This document will represent a detailed section of the Software Design Description Document. The internal structure of Gather, as well as all the major responsibilities of different parts will be described - React dashboard, database organization, assignments review page, AI processing, task management.	Week 12
ALPHA/BETA/Critical Design Review Presentation/ Demonstration	The team will be delivering a presentation of Gather at its current stage of development. It will include showing how all the major software components are functioning together, as well as general functionality of UI. The team will also explain the design and idea behind it, as well as touch on details that need to be fixed or still added.	Week 13
User's manual	The team will deliver the README.md file that will serve as a user's manual for the Gather web application. The manual will explain how to install and manage Gather.	Week 13
Details finalization	The team will finalize the remaining details that needed to be fixed/added after the main demonstration of the Gather application.	Week 14
Final Presentation and Delivery	The team will deliver the final product and present it in class.	Week 15-16

4.2 Project Resources

4.2.1 Hardware Resources

-Team member laptops/desktops capable of running a modern JavaScript development environment.
-Reliable Internet access for cloud services, API access, GitHub, Brightspace, and Firebase.
-LMU development/testing resources as needed for Brightspace registration and institutional API access.

4.2.2 Software Resources

Resource	Planned Use	Notes
React	Frontend user interface	Calendar/list pages, assignment adding/review, settings
Playwright	E2E Testing frontend during development Scraping websites with non static html code.	A way to see how frontend code looks, and testing E2E functionality. Getting raw html data from websites that are dynamic. Cheerio does not work on such websites so this is a necessary alternative.
Node.js + Express	Backend and application API	CRUD endpoints, integrations, processing flow
Pdf-parse	Parse PDF files into raw text	Necessary for making sense of the syllabus.
Cheerio	Extract raw html text from static html websites	Get the textual data from a professors/resources's website to turn into assignments
Firebase	Authentication, database, and temporary file storage	User accounts, assignments, class/integration data
Firebase Local Emulator Suite	Testing backend during development to avoid needing to deploy	E2E Testing
Postman	Manual API endpoint & OAuth debugging	Necessary for testing backend without needing to do it through frontend
Brightspace API	Retrieve authorized user course/assignment data	Requires LMU authorization and user consent
OpenRouter / LLM service	Parse cleaned text into structured assignment data	Model selection may change during development

Git + GitHub	Version control and Software Development Library	Public team repository, all resources committed
Visual Studio Code	Primary code editor	Other editors may be used if repository standards are maintained
Modern browsers	Development and testing	Chrome/Edge/Firefox

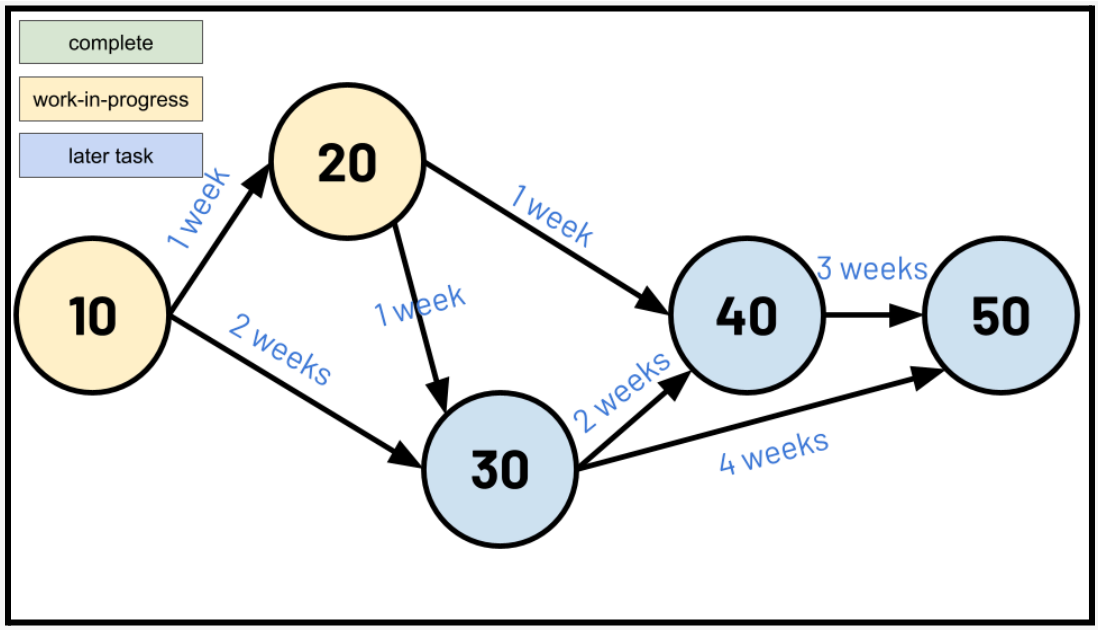
4.3 Project Organization

Ari : Project Lead, Designer, Generalist
Luca : Chief of Backend
Greg : Full-stack Developer
Dan : Chief of Frontend… and FUN!
Ivana : Frontend Developer, Generalist

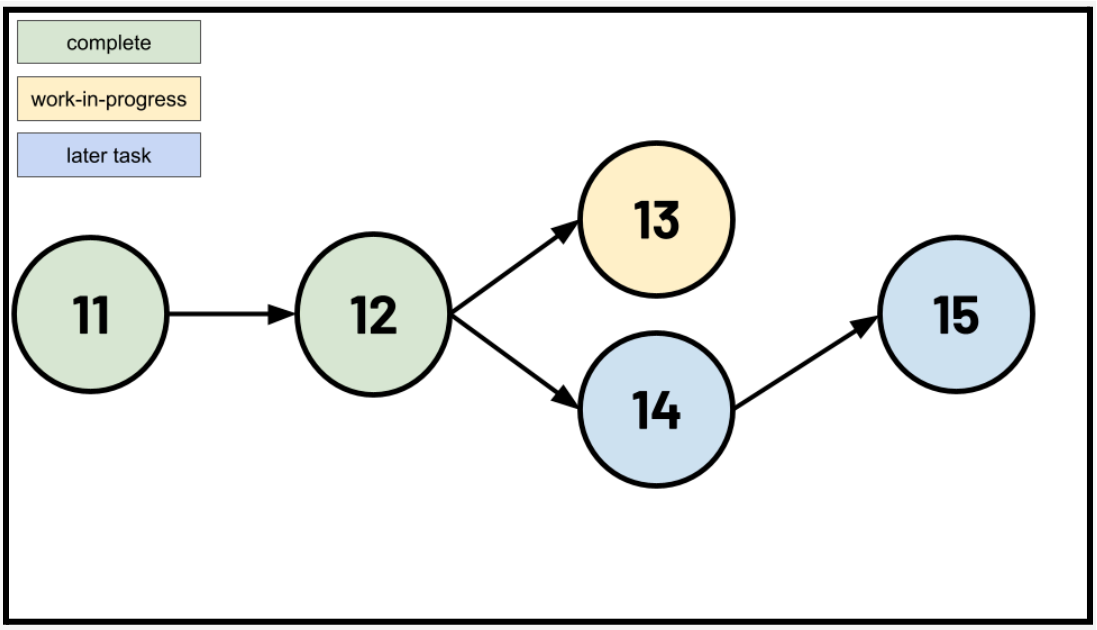
4.4 Project Schedule

4.4.1 PERT Chart

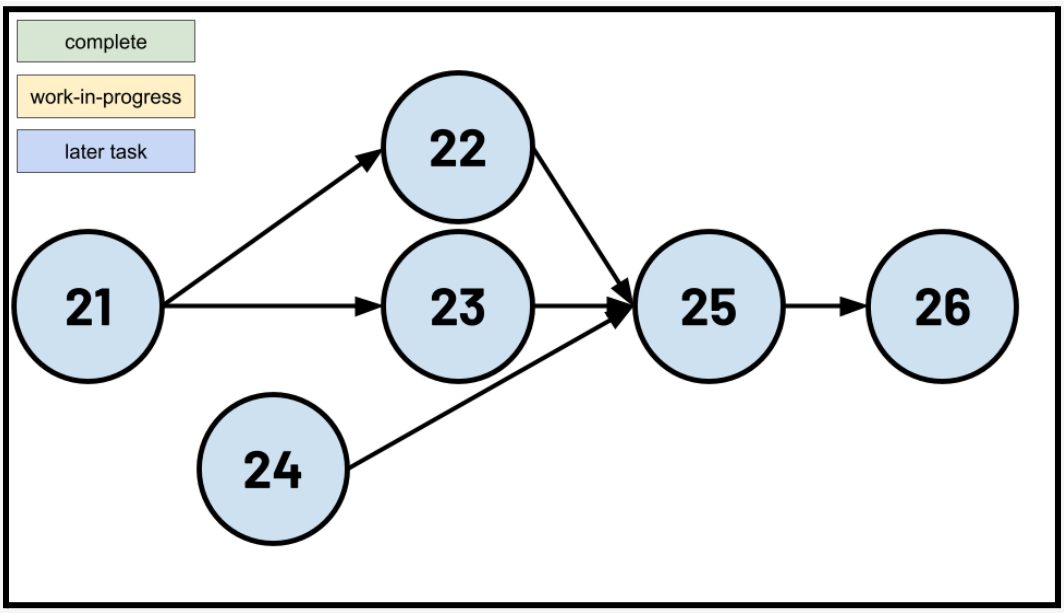
10: Ideate and prepare development environment
20: Create mockups and acquire permissions
30: Build website
40: Build backend
50: Integrate full systems



11: Project proposal (complete)
12: Create software requirements document (complete)
13: Software Development Plan (in-progress)
14: Development environment setup
15: Setup Playwright and Postman accounts

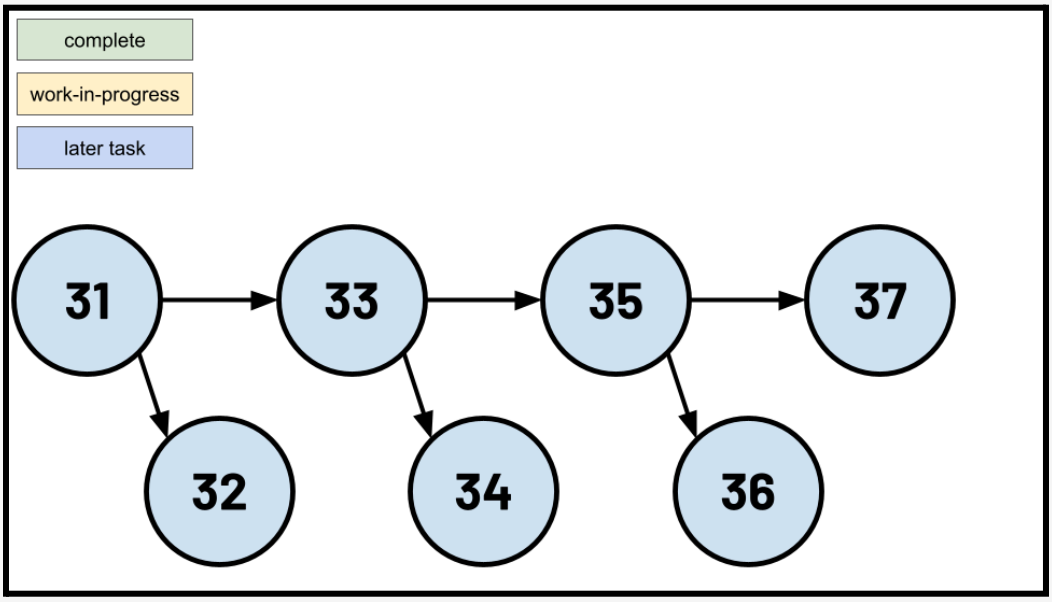


21: Pull website design references
22: Website mockups and design overview
23: Apply critical Firebase modules
24: Request (Brightspace) API access and keys
25: Software Design Description [SDD] architecture section
26: Apply any to-be-used software (Playwright, Postman, etc) to the repository

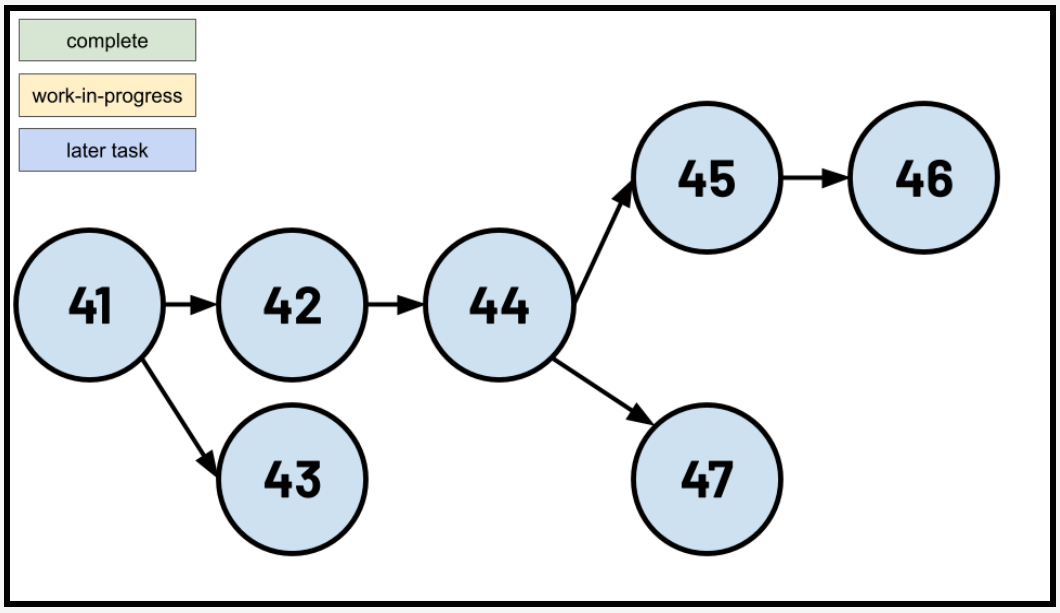


31: Implement layout for main page
32: Implement I/O for main page
33: Implement layout for assignment page

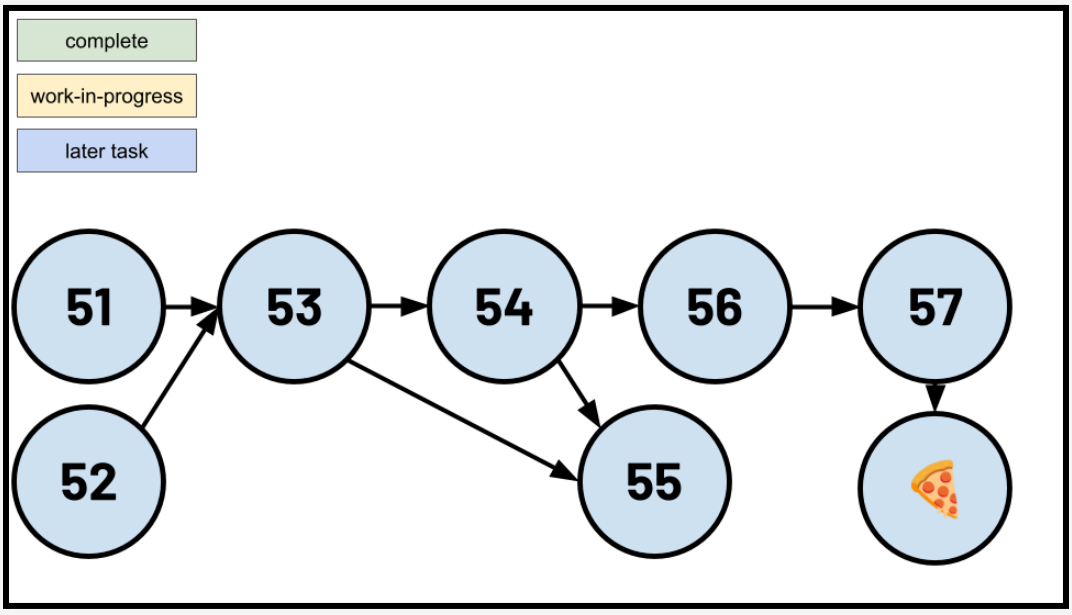
34: Implement I/O for assignment page
35: Implement navigation bar layout
36: Implement navigation bar functionality
37: Implement settings page layout and I/O



41: Receive creation, deletion, edit, and read input from frontend (API creation)
42: Ensure backend can receive Brightspace API calls
43: Setup Firebase Authentication for Google and email
44: Design storage format
45: Ensure Llama formatter (pdf, Brightspace, and html) works
46: Ensure failsafes work (Llama limits, check data format)
47: Apply automatic updates onto active users




```
51: Design stress tests on frontend and DB
52: Design malformed API calls and AI prompts
53: Stress test full-stack with tests
54: Compare current performance to ideal performance from Requirement Specs
55: Optimize to meet specs
56: Finalize documentation
57: Bells, whistles, flourishes
🍕: Pizza party
```



4.4.2 Task / Resource Table

Task	Software	Hardware
User Authentication	-Firebase Auth	-Firebase Cloud Auth Infrastructure
Brightspace API Auth	-OAuth 2.0 -Firebase Cloud Functions	-Google Cloud Serverless Compute
Frontend UI	-React.js -Modern Browsers -Firebase Hosting	-Client End-User Devices -Google Global Edge Network
CRUD operations for the app	-Firebase Cloud Functions -Firebase Firestore	-Google Cloud Platform -Serverless Infrastructure
Testing	-Playwright -Firebase Local Emulator Suite -Postman	-Developer workstation
Turning URLs/PDFs into raw text	-Firebase Cloud Functions -Playwright -Pdf-parse -Cheerio	-Cloud Functions Execution Nodes (Allocated ≥ 1 GiB RAM)
Turning raw text into formatted assignments our backend can use	-Firebase Cloud Functions -OpenRouter / LLM service	-Third-Party LLM Inference Hardware (Remote Cloud)